

```
In [1]: ## importing important libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import warnings

warnings.filterwarnings("ignore")
```

C:\Users\Prashant\anaconda3\lib\site-packages\pandas\core\arrays\masked.py:60: UserWarning: Pandas requires version '1.3.6' or newer of 'bottleneck' (version '1.3.5' currently installed).
from pandas.core import (

```
In [2]: df = pd.read_csv("Travel.csv")
df.head()
```

```
Out[2]:
```

	CustomerID	ProdTaken	Age	TypeofContact	CityTier	DurationOfPitch	Occupation	Gender	NumberOfPersonVisiting	NumberOfFollowu
0	200000	1	41.0	Self Enquiry	3	6.0	Salaried	Female	3	
1	200001	0	49.0	Company Invited	1	14.0	Salaried	Male	3	
2	200002	1	37.0	Self Enquiry	1	8.0	Free Lancer	Male	3	
3	200003	0	33.0	Company Invited	1	9.0	Salaried	Female	2	
4	200004	0	NaN	Self Enquiry	1	8.0	Small Business	Male	2	

Data Cleaning

Handling Missing values

1. Handling Missing values
2. Handling Duplicates
3. Check data type

4. Understand the dataset

```
In [3]: df.isnull().sum()
```

```
Out[3]: CustomerID          0
        ProdTaken          0
        Age              226
        TypeofContact     25
        CityTier          0
        DurationOfPitch    251
        Occupation        0
        Gender            0
        NumberOfPersonVisiting 0
        NumberOfFollowups  45
        ProductPitched     0
        PreferredPropertyStar 26
        MaritalStatus      0
        NumberOfTrips      140
        Passport           0
        PitchSatisfactionScore 0
        OwnCar             0
        NumberOfChildrenVisiting 66
        Designation        0
        MonthlyIncome      233
        dtype: int64
```

```
In [4]: ### Check all the categories
        df['Gender'].value_counts()
```

```
Out[4]: Gender
        Male      2916
        Female    1817
        Fe Male    155
        Name: count, dtype: int64
```

```
In [5]: df['MaritalStatus'].value_counts()
```

```
Out[5]: MaritalStatus
        Married    2340
        Divorced   950
        Single     916
        Unmarried   682
        Name: count, dtype: int64
```

```
In [6]: df['TypeofContact'].value_counts()
```

```
Out[6]: TypeofContact
Self Enquiry      3444
Company Invited   1419
Name: count, dtype: int64
```

```
In [7]: df['Gender'] = df['Gender'].replace('Fe Male', 'Female')
df['MaritalStatus'] = df['MaritalStatus'].replace('Single', 'Unmarried')
```

```
In [8]: ### Check all the categories
df['Gender'].value_counts()
```

```
Out[8]: Gender
Male      2916
Female    1972
Name: count, dtype: int64
```

```
In [9]: df.head()
```

```
Out[9]:
```

	CustomerID	ProdTaken	Age	TypeofContact	CityTier	DurationOfPitch	Occupation	Gender	NumberOfPersonVisiting	NumberOfFollowu
0	200000	1	41.0	Self Enquiry	3	6.0	Salaried	Female	3	
1	200001	0	49.0	Company Invited	1	14.0	Salaried	Male	3	
2	200002	1	37.0	Self Enquiry	1	8.0	Free Lancer	Male	3	
3	200003	0	33.0	Company Invited	1	9.0	Salaried	Female	2	
4	200004	0	NaN	Self Enquiry	1	8.0	Small Business	Male	2	

Imputing Null values

1. Impute Median value for Age column
2. Impute Mode for Type of Contract
3. Impute Median for Duration of Pitch
4. Impute Mode for NumberofFollowup as it is Discrete feature

5. Impute Mode for PreferredPropertyStar
6. Impute Median for NumberofTrips
7. Impute Mode for NumberOfChildrenVisiting
8. Impute Median for MonthlyIncome

```
In [10]: #Age
df.Age.fillna(df.Age.median(), inplace=True)

#TypeofContract
df.TypeofContract.fillna(df.TypeofContract.mode()[0], inplace=True)

#DurationOfPitch
df.DurationOfPitch.fillna(df.DurationOfPitch.median(), inplace=True)

#NumberOfFollowups
df.NumberOfFollowups.fillna(df.NumberOfFollowups.mode()[0], inplace=True)

#PreferredPropertyStar
df.PreferredPropertyStar.fillna(df.PreferredPropertyStar.mode()[0], inplace=True)

#NumberOfTrips
df.NumberOfTrips.fillna(df.NumberOfTrips.median(), inplace=True)

#NumberOfChildrenVisiting
df.NumberOfChildrenVisiting.fillna(df.NumberOfChildrenVisiting.mode()[0], inplace=True)

#MonthlyIncome
df.MonthlyIncome.fillna(df.MonthlyIncome.median(), inplace=True)
```

```
In [11]: df.isnull().sum()
```

```
Out[11]: CustomerID      0
         ProdTaken      0
         Age            0
         TypeofContact  0
         CityTier       0
         DurationOfPitch 0
         Occupation     0
         Gender         0
         NumberOfPersonVisiting 0
         NumberOfFollowups 0
         ProductPitched 0
         PreferredPropertyStar 0
         MaritalStatus  0
         NumberOfTrips  0
         Passport       0
         PitchSatisfactionScore 0
         OwnCar         0
         NumberOfChildrenVisiting 0
         Designation    0
         MonthlyIncome  0
         dtype: int64
```

```
In [12]: df.drop('CustomerID', inplace=True, axis=1)
```

Feature Engineering

Feature Extraction

```
In [ ]: df.head()
```

```
In [ ]: # create new column for feature
df['TotalVisiting'] = df['NumberOfPersonVisiting'] + df['NumberOfChildrenVisiting']
df.drop(columns=['NumberOfPersonVisiting', 'NumberOfChildrenVisiting'], axis=1, inplace=True)
```

```
In [13]: ## get all the numeric features
num_features = [feature for feature in df.columns if df[feature].dtype != 'O']
print('Num of Numerical Features :', len(num_features))
```

Num of Numerical Features : 13

```
In [14]: ##categorical features
cat_features = [feature for feature in df.columns if df[feature].dtype == 'O']
```

```
print('Num of Categorical Features :', len(cat_features))
```

Num of Categorical Features : 6

```
In [15]: ## Discrete features
discrete_features=[feature for feature in num_features if len(df[feature].unique())<=25]
print('Num of Discrete Features :',len(discrete_features))
```

Num of Discrete Features : 10

```
In [16]: ## coontinuous features
continuous_features=[feature for feature in num_features if feature not in discrete_features]
print('Num of Continuous Features :',len(continuous_features))
```

Num of Continuous Features : 3

```
In [ ]: df.head()
```

```
In [17]: from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
for i in cat_features:
    df[i]=le.fit_transform(df[i])
```

```
In [18]: df.head()
```

```
Out[18]:
```

	ProdTaken	Age	TypeofContact	CityTier	DurationOfPitch	Occupation	Gender	NumberOfPersonVisiting	NumberOfFollowups	ProductP
0	1	41.0	1	3	6.0	2	0	3	3.0	
1	0	49.0	0	1	14.0	2	1	3	4.0	
2	1	37.0	1	1	8.0	0	1	3	4.0	
3	0	33.0	0	1	9.0	2	0	2	3.0	
4	0	36.0	1	1	8.0	3	1	2	3.0	

```
In [20]: from scipy.stats import zscore
z=np.abs(zscore(df))
df1=df[(z<3).all(axis=1)]
df=df1.copy()
```

Train Test Split And Model Training

```
In [21]: from sklearn.model_selection import train_test_split
X = df.drop(['ProdTaken'], axis=1)
y = df['ProdTaken']
```

```
In [22]: df.shape
```

```
Out[22]: (4876, 19)
```

```
In [23]: from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
X_scale=sc.fit_transform(X)
```

```
In [26]: # separate dataset into train and test
X_train, X_test, y_train, y_test = train_test_split(X_scale,y,test_size=0.2,random_state=42)
X_train.shape, X_test.shape
```

```
Out[26]: ((3900, 18), (976, 18))
```

```
In [27]: from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
```

```
In [28]: dtc=DecisionTreeClassifier()
dtc.fit(X_train,y_train)
y_pred=dtc.predict(X_test)
print(classification_report(y_test,y_pred))
```

	precision	recall	f1-score	support
0	0.96	0.95	0.95	811
1	0.75	0.78	0.77	165
accuracy			0.92	976
macro avg	0.85	0.87	0.86	976
weighted avg	0.92	0.92	0.92	976

```
In [29]: rfc=RandomForestClassifier()
rfc.fit(X_train,y_train)
y_pred=rfc.predict(X_test)
print(classification_report(y_test,y_pred))
```

	precision	recall	f1-score	support
0	0.95	1.00	0.97	811
1	0.97	0.72	0.83	165
accuracy			0.95	976
macro avg	0.96	0.86	0.90	976
weighted avg	0.95	0.95	0.95	976

```
In [ ]: ## Hyperparameter Training
rf_params = {"max_depth": [5, 8, 15, None, 10],
             "max_features": [5, 7, "auto", 8],
             "min_samples_split": [2, 8, 15, 20],
             "n_estimators": [100, 200, 500, 1000]}
```

```
In [ ]: rf_params
```

```
In [ ]: from sklearn.model_selection import GridSearchCV
```

```
In [ ]: GCV = GridSearchCV(RandomForestClassifier(),rf_params)
GCV.fit(X_train,y_train)
```

```
In [ ]: GCV.best_params_
```

```
In [ ]: rfc=RandomForestClassifier(-- )
rfc.fit(X_train,y_train)
y_pred=dtc.predict(X_test)
print(classification_report(y_test,y_pred))
```

```
In [ ]:
```

```
In [ ]:
```